
浙江工业大学

C++程序设计课程设计 课设报告

2024/2025(2)



实验题目 职工工资管理系统

学生姓名 田笑喧

学生学号 302023572097

学生班级 软件工程 2304

任课教师 毛国红

提交日期 2025 年 6 月 25 日

计算机科学与技术学院

职工工资管理系统 实验报告

一、 大型实验的内容

职工工资管理系统，用于进行职工工资的管理，要求完成的主要的功能包括：

- 1、设计菜单实现功能选择；
- 2、输入功能：输入职工信息，并保存到文件中；
- 3、查询功能：
 - 1) 能够根据工号精确查询职工信息；
 - 2) 能够根据姓名、科室精确或者模糊查询职工信息；
 - 3) 能根据科室查询职工信息并输出；
 - 4) 按科室进行工资统计，计算各科室的平均工资；
- 4、根据职工的工资排序输出；
- 5、根据工号修改职工信息；
- 6、根据工号删除职工信息；
- 7、所有的增加、修改、删除能同步到文件；也从文件读取数据到程序。
- 8、考虑各项数据的合法性检查

要求使用学习过的 C/C++程序设计的知识完成职工工资管理系统的设计与实现。

二、 运行环境

职工工资管理系统在 Visual Studio Code 平台下开发，操作系统：Windows 11。

硬件环境：

处理器：13th Gen Intel(R) Core(TM) i7-13620H 2.40 GHz

内存：16.00GB

系统类型：64 位操作系统

三、 实验课题分析

3.1 职工工资管理系统的主要功能

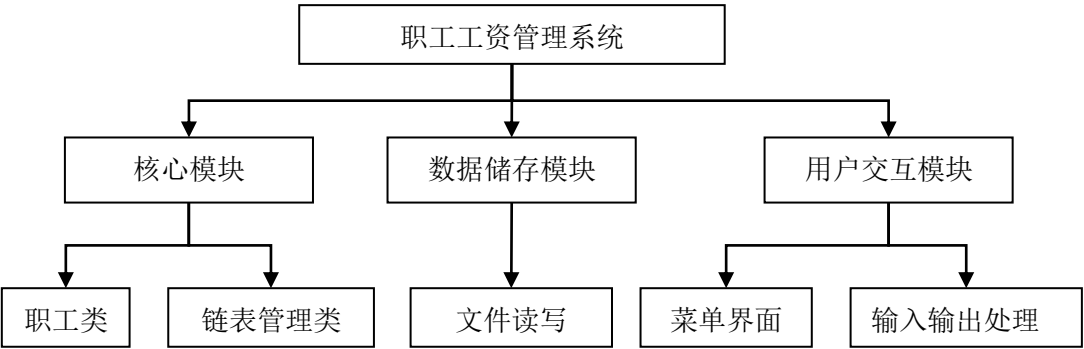


图 1 系统总体结构

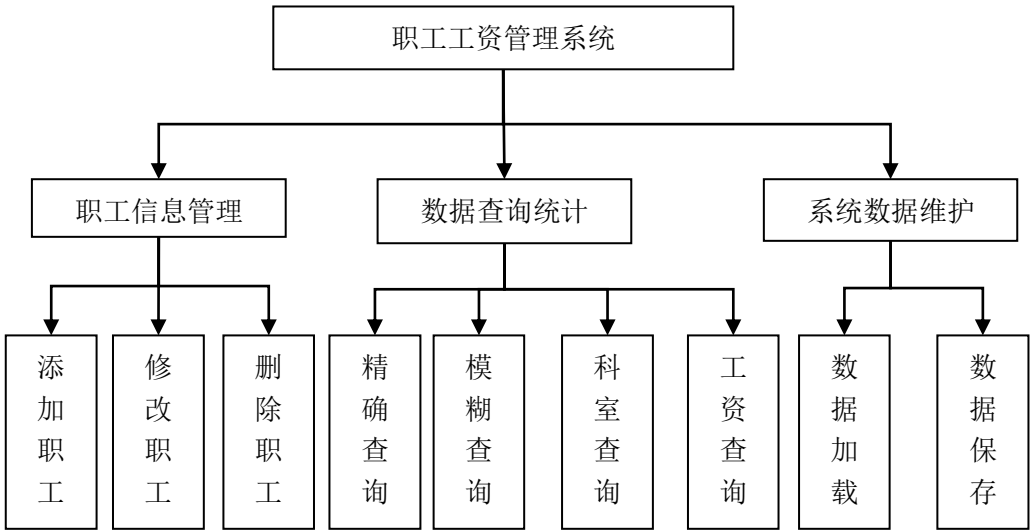


图 2 系统的主要功能

系统各模块的功能具体描述为：

职工信息管理

1 添加新职工

选录入新职工的全部信息，包括姓名、工号、联系方式等
系统会自动检查工号是否重复，确保唯一性
添加成功后立即生效并永久保存

2、修改职工信息

通过工号快速找到要修改的职工
可以修改任意信息项（如调整工资、更换科室等）
修改时自动保留原有正确信息

3、删除离职职工

通过工号精确删除指定职工记录

删除后相关信息将彻底清除

信息查询服务

4、精确查询

- 按工号快速查找特定职工
- 按姓名精确查找同名职工
- 按科室精确查找特定职工
- 立即显示完整个人信息

5、模糊查询

- 输入姓名关键字查找相关职工
- 支持查找结果列表展示

6、科室统计

- 按科室精确查找特定职工
- 输入科室关键字筛选相关人员

7、工资查询

- 按工资从低到高排序输出
- 按科室输出平均工资
- 系统数据维护

8、数据加载

- 系统每次启动时，自动从数据文件加载所有职工信息
- 无需手动操作即可获取最新数据
- 加载过程在后台静默完成

9、数据保存

- 添加、修改或删除职工信息后立即自动保存
- 无需手动执行保存操作
- 避免因忘记保存导致数据丢失

3.2 系统分析及设计

系统涉及对象有个基本类：职工类(Employee)、链表节点类(EmployeeNode)和链表管理类(EmployeeList)。三个核心类及其角色如表 1 所示：

表 1 三个核心类及其角色

类名	类型	职责	与其他类关系
Employee	数据类	存储职工基本信息（姓名、工号等）	被 EmployeeNode 包含
EmployeeNode	节点类	组成链表的基本单元，包含 Employee 数据和指向	包含 Employee，被 EmployeeList

		下一个节点的指针	管理
EmployeeList	管理类	封装链表操作（增删改查）、文件 I/O 和统计功能	管理 EmployeeNode 链表

类关系图解如图 2 所示。

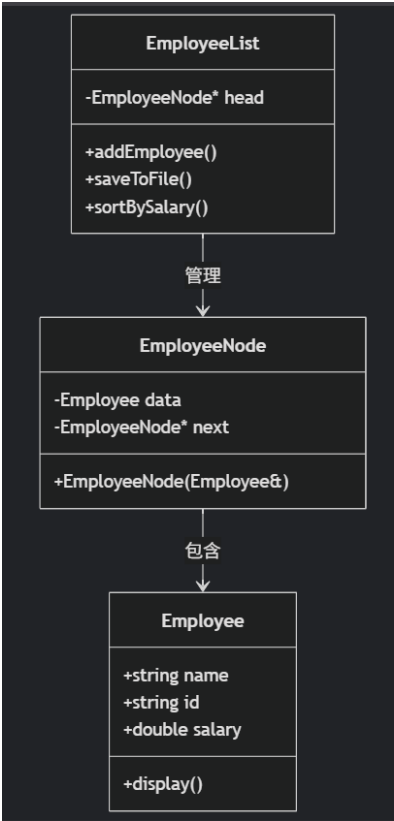


图 2 类关系图解

用文本文件进行数据的保存，需要保存的数据包括姓名，性别，工号，电话，科室，工资。

3.3 系统的实现

(1) 类的编写

包含了职工类(Employee)、链表节点类(EmployeeNode)和链表管理类(EmployeeList)。具体类结构声明如下：

● Employee 类：

```
class Employee {
private:
    string name;           // 姓名
```

```
string gender;      // 性别
string id;          // 工号
string phone;       // 电话
string department;  // 所在科室
double salary;      // 工资

public:
    // 构造函数
    Employee();
    Employee(string n, string g, string i, string p, string d, double s);

    // 获取和设置成员变量的方法
    string getName() const;
    void setName(const string &n);

    string getGender() const;
    void setGender(const string &g);

    string getId() const;
    void setId(const string &i);

    string getPhone() const;
    void setPhone(const string &p);

    string getDepartment() const;
    void setDepartment(const string &d);

    double getSalary() const;
    void setSalary(double s);

    // 显示职工信息
    void display() const;

    // 重载比较运算符，用于排序
    bool operator<(const Employee &other) const;
```

```
};
```

(2) 链表的使用

系统实现采用文件的输入输出流对文本数据进行读取与写入，但是由于姓名，性别，工号，电话，科室，工资都是一个数据的集合，于是对数据的存储组织使用了单向链表。

因为职工工资管理系统在查询、修改、添加的时候都需要处理大量的数据，所以使用链表十分必要。在 `Employee` 类的基础上定义一个对应的 `EmployeeList` 类和 `EmployeeNode` 类来管理员工数据，具体的结构声明如下：

● `EmployeeList` 类

```
class EmployeeList {
private:
    EmployeeNode *head;
    int size;

    // 从文件中读取数据
    void loadFromFile();
    // 保存数据到文件
    void saveToFile();

public:
    EmployeeList();
    ~EmployeeList();

    // 添加职工
    void addEmployee(const Employee &emp);
    // 根据工号查找职工
    Employee* findEmployeeById(const string &id);
    // 根据姓名查找职工（精确或模糊）
    vector<Employee*> findEmployeesByName(const string &name, bool exactMatch
= true);
    // 根据科室查找职工（精确或模糊）
    vector<Employee*> findEmployeesByDepartment(const string &dept, bool
exactMatch = true);
    // 根据工号删除职工
    bool deleteEmployeeById(const string &id);
    // 根据工号修改职工信息
```

```
bool updateEmployeeById(const string &id, const Employee &newData);  
// 按工资排序  
void sortBySalary();  
// 计算科室平均工资  
void calculateDepartmentAvgSalary();  
// 显示所有职工  
void displayAll() const;  
// 清空链表  
void clear();  
  
int getSize() const;  
};
```

- **EmployeeNode 类:**

```
class EmployeeNode {  
public:  
    Employee data;  
    EmployeeNode *next;  
  
    EmployeeNode(const Employee &emp);  
};
```

在运用时，令当前职工的 `next` 结点指向新的结点，即结点的指针 `next` 保存新的职工结点的地址（如下图 3 所示），以此类推，所有职工信息就通过链表的形式串联起来了。

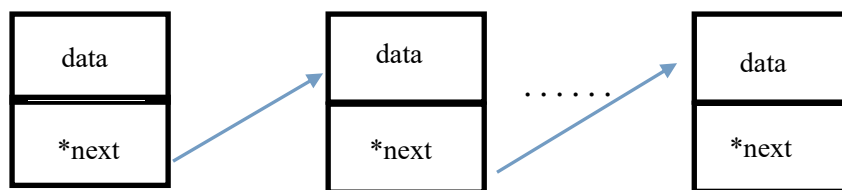


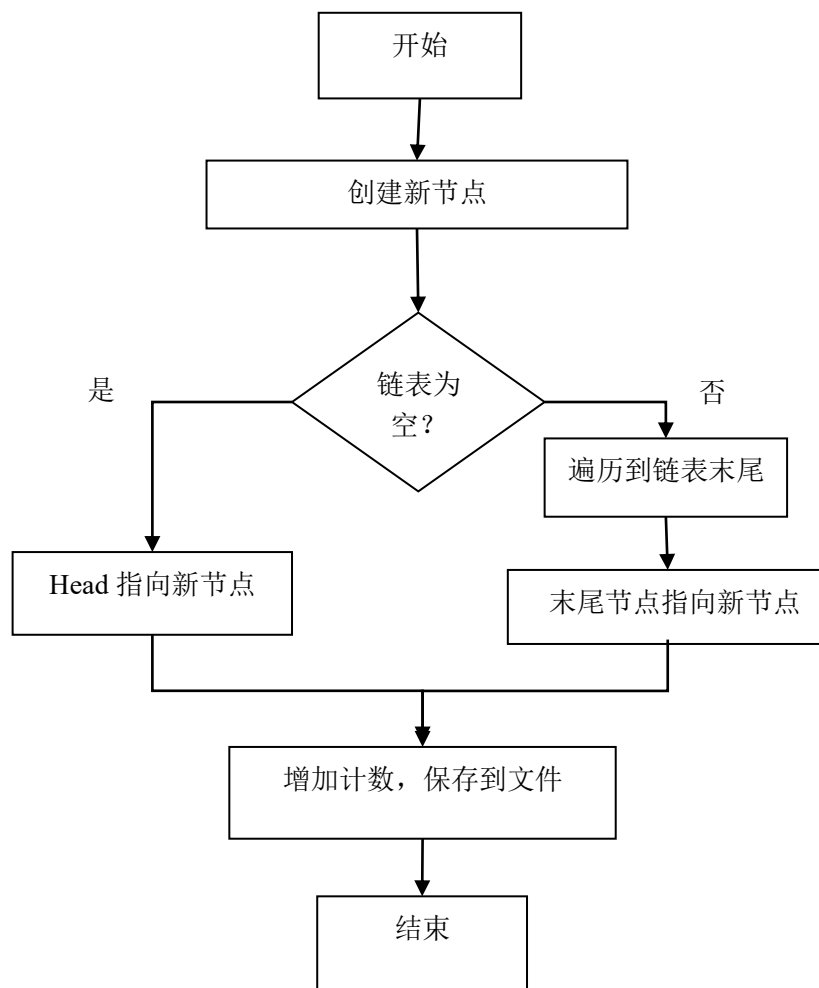
图 3 链表的建立

- **职工的添加：
界面：**


```
--- 添加职工 ---  
请输入职工信息：  
姓名：田笑喧  
性别：女  
工号：3020235720+7  
电话：15356678309  
科室：计算机  
工资：123654'  
职工添加成功！
```

描述：

首先检查输入职工信息的合法性，包括工号唯一性、电话格式和工资有效性，然后动态创建一个新的 `EmployeeNode` 节点存储职工数据。若链表为空，直接将头指针指向新节点；若非空，则从头节点开始遍历直到找到最后一个节点（其 `next` 指针为 `nullptr` 的节点），将新节点链接到链表末尾。完成插入后，链表长度计数器递增，并立即调用 `saveToFile()` 将整个链表的最新状态以二进制形式写入数据文件，确保数据持久化。

流程图：

- 职工的删除:

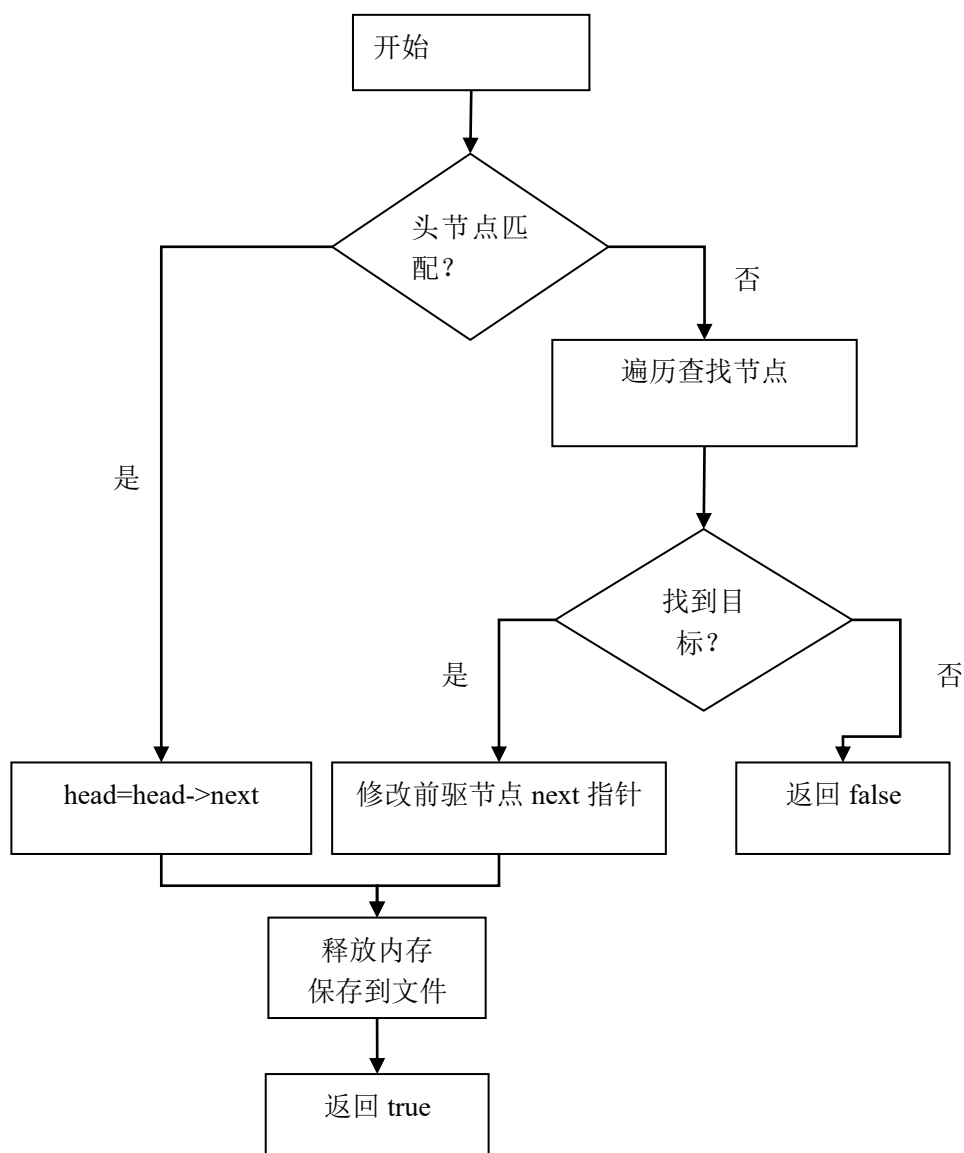
界面:

```
--- 删除职工信息 ---  
请输入要删除的职工工号: 302023572097  
职工删除成功!
```

描述:

首先处理链表为空或头节点即为目标节点的特殊情况, 直接调整 head 指针并释放内存。对于一般情况, 维护一个前驱节点指针, 遍历过程中比对工号, 找到目标节点后修改前驱节点的 next 指针以跳过待删除节点, 然后释放目标节点内存。无论是否找到目标, 操作后都立即同步更新数据文件, 并返回操作状态布尔值。

流程图:



- 职工的精确查询 (按工号/姓名):

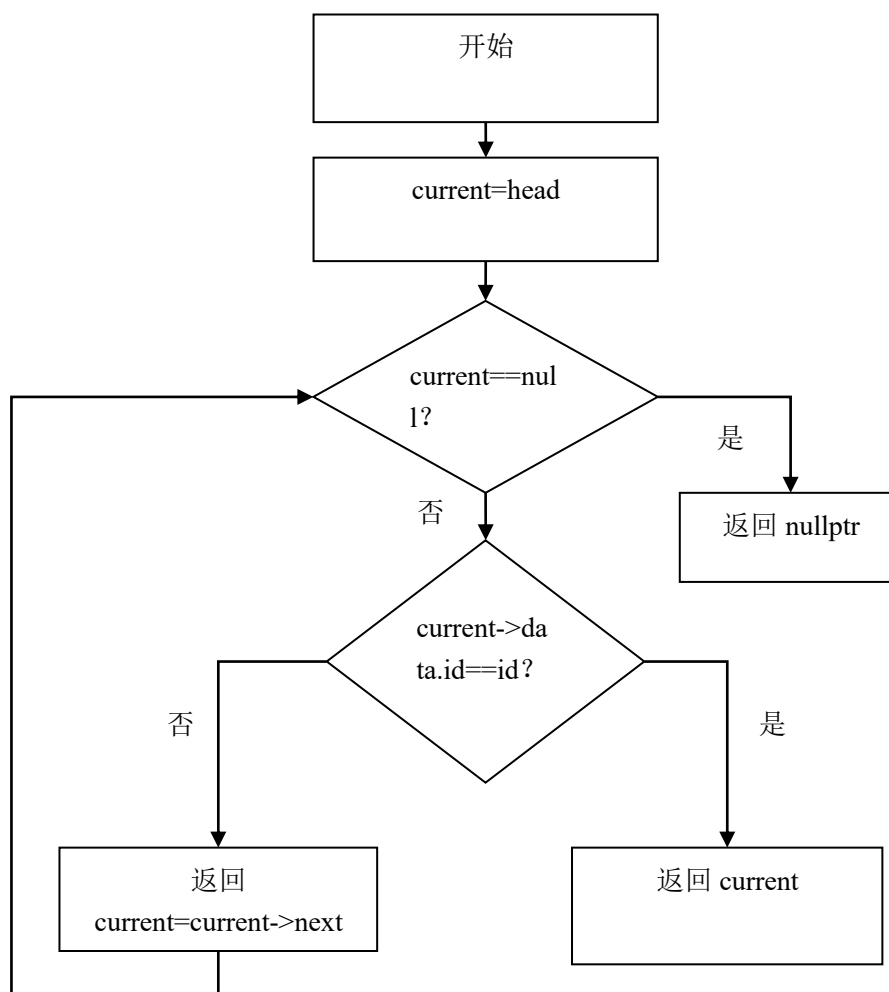
界面:

```
--- 查询职工 ---
1. 按工号查询
2. 按姓名查询(精确)
3. 按姓名查询(模糊)
请选择查询方式: 1
请输入工号: 3020235720+7
工号    姓名    性别    电话    科室    工资
-----
3020235720+7 田笑喧 女    15356678309    计算机    123654.00
```

描述:

从头节点开始遍历链表，依次访问每个节点的 data 成员，将目标工号与当前节点存储的职工工号进行比对。若找到匹配的工号，立即返回指向该节点数据的指针；若遍历至链表末尾仍未找到匹配项，则返回空指针。整个过程严格遵循线性查找逻辑，时间复杂度与链表长度成正比。

流程图:

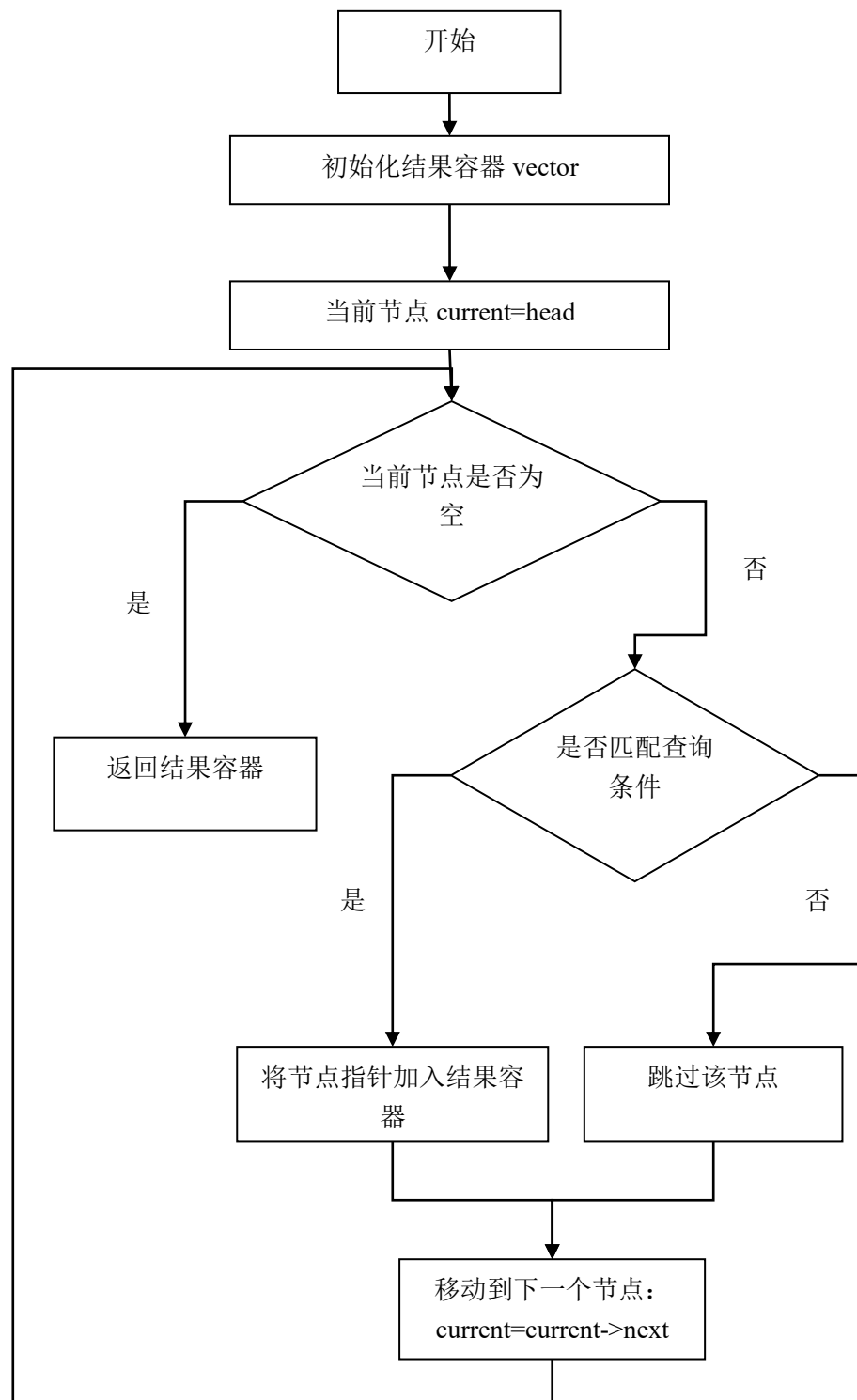


- 模糊查询（按姓名/科室）:

描述:

首先处理链表为空或头节点即为目标节点的特殊情况，直接调整 `head` 指针并释放内存。对于一般情况，维护一个前驱节点指针，遍历过程中比对工号，找到目标节点后修改前驱节点的 `next` 指针以跳过待删除节点，然后释放目标节点内存。无论是否找到目标，操作后都立即同步更新数据文件，并返回操作状态布尔值。

流程图:



- 工资排序：
界面：

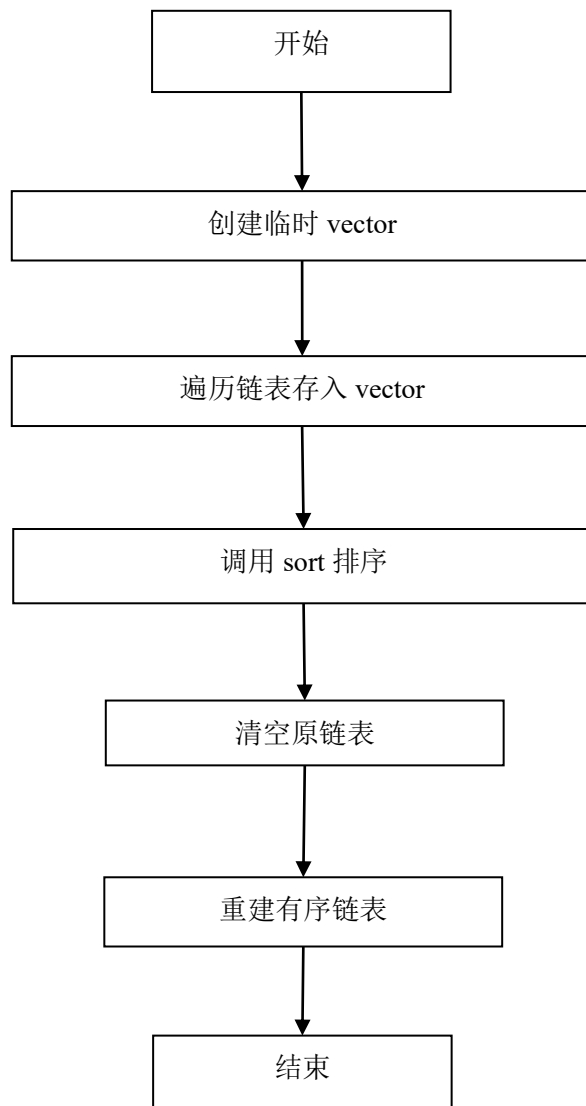
--- 按工资排序输出 ---

工号	姓名	性别	电话	科室	工资
3020235720	田笑喧	女	15356678309	计算机	123654.00
123456789621	xx	女	15656678309	软工	987654.00

描述:

创建临时 vector 容器，遍历链表将所有职工数据拷贝到 vector 中，利用 STL 的 sort 算法按工资排序（依赖 Employee 类的<运算符重载）。清空原链表后，按排序后的顺序重新构建链表，此过程保持数据完整性且不改变原始文件存储顺序，仅影响内存中的链表结构。

流程图:



- 科室平均工资:
界面:

```
--- 科室工资统计 ---
```

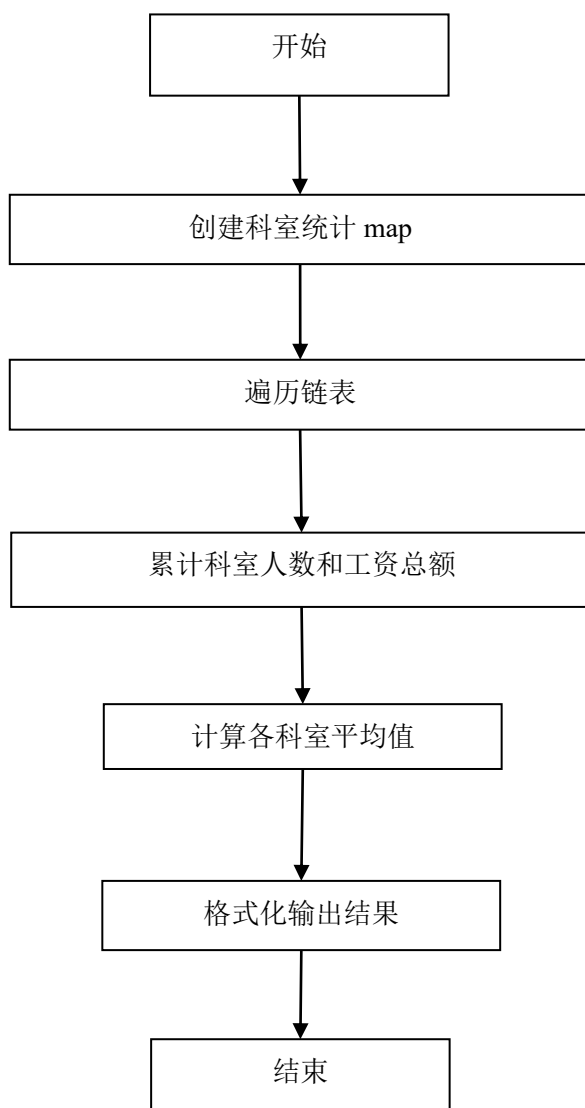
```
科室工资统计：
```

```
-----  
科室      人数      平均工资  
计算机      1      123654.00  
软工        1      987654.00
```

描述：

使用 `map<string, pair<int, double>>` 数据结构，键为科室名称，值为人数和工资总和的二元组。遍历链表时，对每个节点按其科室分类累加人数和工资总额。遍历完成后，计算每个科室的平均工资（工资总额/人数），最后以表格形式输出科室名称、人数和平均工资三项统计结果。

流程图：



- 显示所有职工：

界面:

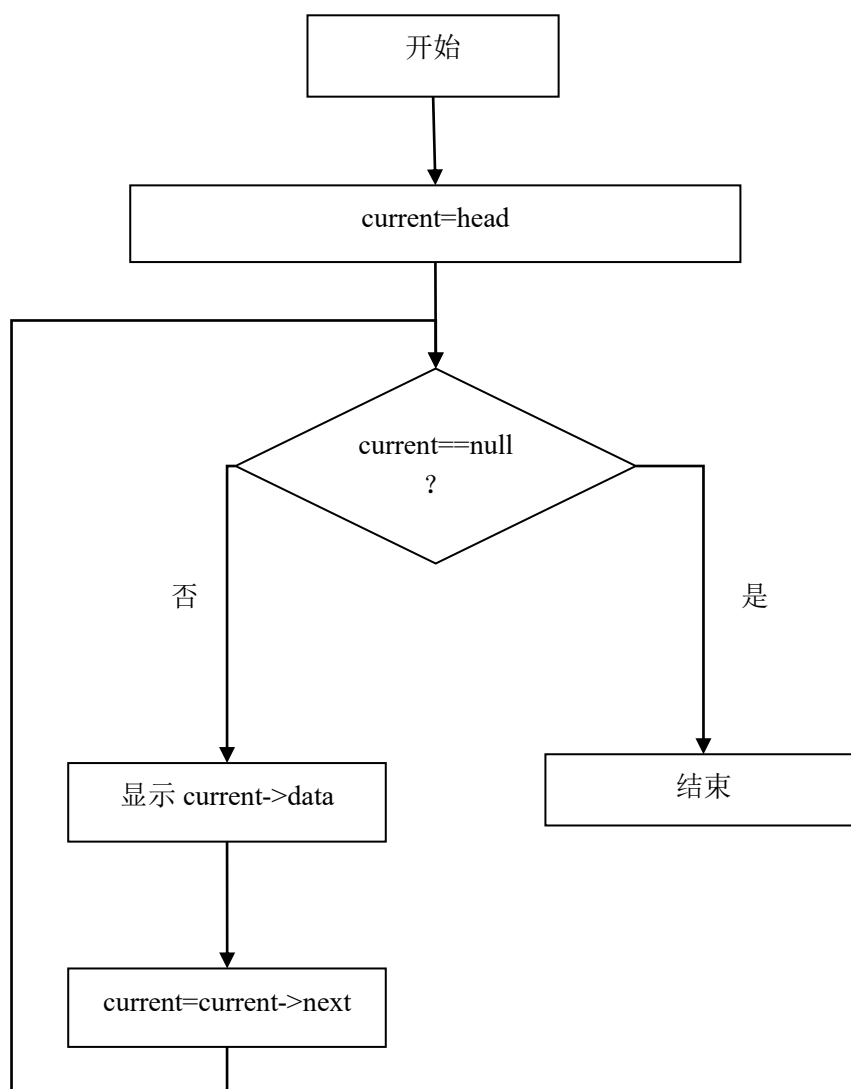
--- 显示所有职工 ---						
工号	姓名	性别	电话	科室	工资	

3020235720+7	田笑喧	女	15356678309	计算机	123654.00	
123456789621t	xx	女	15656678309	软工	987654.00	

描述:

首先打印表头信息（各字段名称），然后从头节点开始遍历链表。对于每个节点，调用其 data 成员的 display()方法，该方法按照预定格式对齐输出各字段值。遍历完成后，若链表为空则显示相应提示信息。输出过程不修改链表结构，仅为只读操作。

流程图:



(3) 交互界面以及登录菜单的实现

系统运行开始的界面如图 5 所示:


```
文件不存在，将创建新文件。
=====
      欢迎使用职工工资管理系统
=====
系统已启动，已从文件加载 0 条职工记录
请根据菜单选择操作：

职工工资管理系统
1. 添加职工
2. 查询职工
3. 按科室查询职工
4. 按工资排序输出
5. 修改职工信息
6. 删除职工信息
7. 显示所有职工
8. 科室工资统计
0. 退出系统
请选择操作：请输入选项数字(0-8): 
```

图 5 开始界面

主要通过选择结构和循环结构实现界面的前进和后退。例如，第二个查询界面出现 3 个选择：1. 按工号查询 2. 按姓名查询(精确) 3. 按姓名查询(模糊)，选择之后转到下一个界面。

四、 实验调试、测试、运行记录及分析

主要的测试经过如下：

```
文件不存在，将创建新文件。
=====
      欢迎使用职工工资管理系统
=====
系统已启动，已从文件加载 0 条职工记录
请根据菜单选择操作：

职工工资管理系统
1. 添加职工
2. 查询职工
3. 按科室查询职工
4. 按工资排序输出
5. 修改职工信息
6. 删除职工信息
7. 显示所有职工
8. 科室工资统计
0. 退出系统
请选择操作：请输入选项数字(0-8): 
```

在开始界面输入“1”即可跳转到添加职工界面。

```
--- 添加职工 ---
请输入职工信息：
姓名：田笑喧
性别：女
工号：302023572097
电话：15356678309
科室：软工
工资：123456
职工添加成功！
```

然后输入“2”，屏幕上跳出查询职工界面。

```
--- 查询职工 ---
1. 按工号查询
2. 按姓名查询(精确)
3. 按姓名查询(模糊)
请选择查询方式：█
```

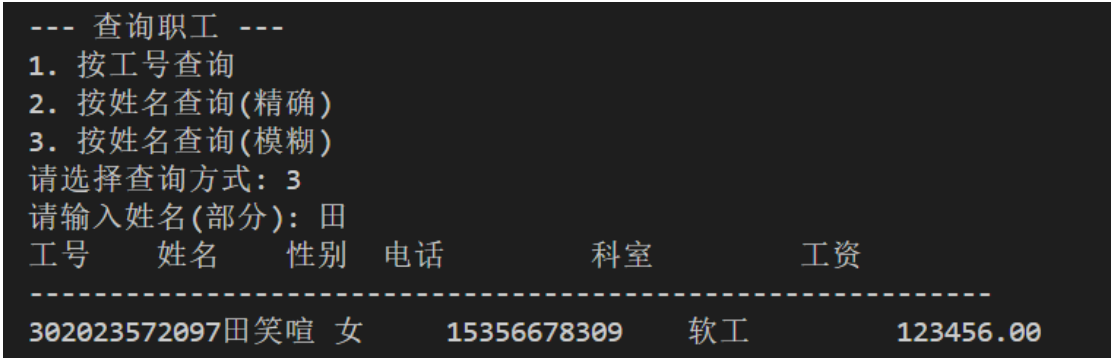
1. 按工号查询

```
--- 查询职工 ---
1. 按工号查询
2. 按姓名查询(精确)
3. 按姓名查询(模糊)
请选择查询方式：1
请输入工号：302023572097
工号      姓名      性别      电话      科室      工资
-----
302023572097田笑喧 女      15356678309      软工      123456.00
```

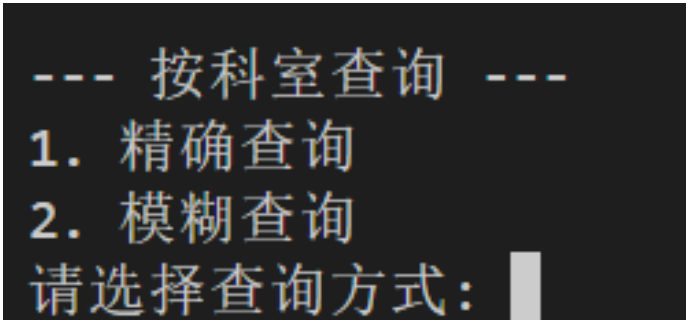
2. 按姓名查询（精确）

```
--- 查询职工 ---
1. 按工号查询
2. 按姓名查询(精确)
3. 按姓名查询(模糊)
请选择查询方式：2
请输入姓名：田笑喧
工号      姓名      性别      电话      科室      工资
-----
302023572097田笑喧 女      15356678309      软工      123456.00
```

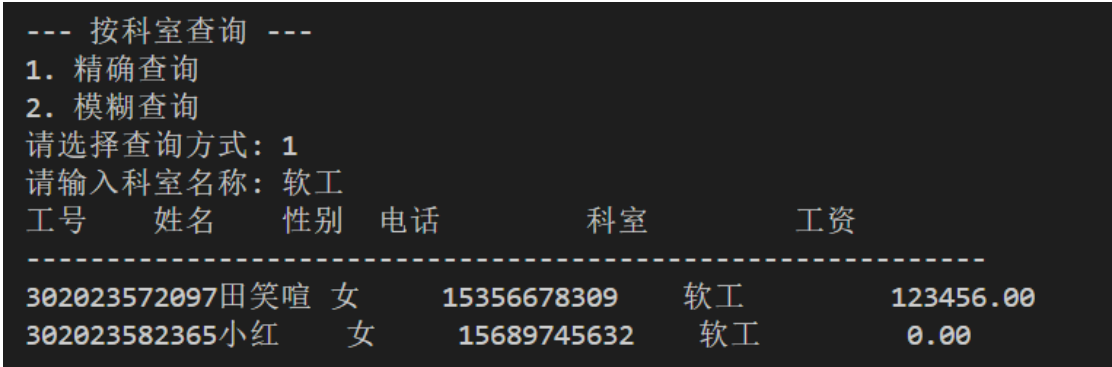
3. 按姓名查询（模糊）



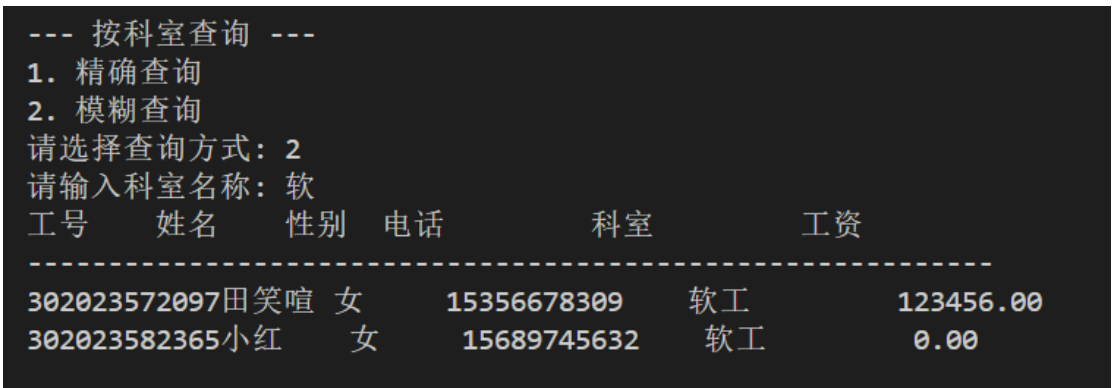
然后输入“3”，屏幕上跳出按科室查询职工界面。



1. 精确查询



2. 模糊查询



然后输入“4”，屏幕上跳出按工资排序输出界面。

--- 按工资排序输出 ---

工号	姓名	性别	电话	科室	工资
302023582365	小红	女	15689745632	软工	0.00
302023572097	田笑喧	女	15356678309	软工	123456.00
302023573058	小明	男	19858122990	计算机	123698.00

然后输入“5”，屏幕上跳出修改职工信息界面。

--- 修改职工信息 ---

请输入要修改的职工工号：302023582365

当前职工信息：

工号	姓名	性别	电话	科室	工资
302023582365	小红	女	15689745632	软工	0.00

请输入新的职工信息(不修改请直接回车)：

姓名(小红)：小蓝

性别(女)：

工号(302023582365)：

电话(15689745632)：

科室(软工)：

工资(0.00)：

职工信息修改成功！

然后输入“6”，屏幕上跳出删除职工信息界面。

--- 删除职工信息 ---

请输入要删除的职工工号：302023573058

职工删除成功！

然后输入“7”，屏幕上跳出显示所有职工信息界面。

--- 显示所有职工 ---

工号	姓名	性别	电话	科室	工资
302023582365	小蓝	女	15689745632	软工	0.00
302023572097	田笑喧	女	15356678309	软工	123456.00

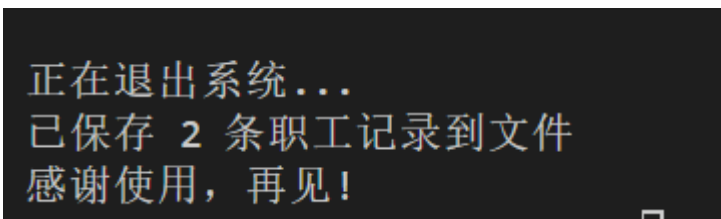
然后输入“8”，屏幕上跳出科室工资统计界面。

--- 科室工资统计 ---

科室工资统计：

科室	人数	平均工资
软工	2	61728.00

然后输入“0”，屏幕上跳出退出系统提示界面。



遇到的问题及解决方法如下：

● 问题 1:

问题描述：程序运行时，删除职工信息后，再次执行查询或遍历操作时出现内存访问异常。在删除节点时，仅修改了链表指针关系，但未正确释放节点内存，导致：内存泄漏（被删除的节点未被 `delete` 释放）或悬空指针（其他代码可能误访问已删除的节点）

```
void deleteEmployeeById(string id) {
    EmployeeNode* current = head;
    while (current) {
        if (current->data.getId() == id) {
            current = current->next; // 错误！仅移动指针，未释放内存
            size--;
            return;
        }
        current = current->next;
    }
}
```

解决方法：首先判断链表是否为空，若为空则直接返回删除失败。对于非空链表，需区分两种情况处理：一是当删除的节点为头节点时，暂存头节点指针，将头指针后移一位，再释放原头节点内存并更新链表长度；二是当删除中间或尾部节点时，从头节点开始遍历，使用前驱指针记录当前位置，在找到目标节点后，先将前驱节点的 `next` 指针指向目标节点的下一个节点，确保链表不断裂，再安全释放目标节点内存。

```
bool EmployeeList::deleteEmployeeById(const string &id) {
    if (head == nullptr) return false;

    if (head->data.getId() == id) {
        EmployeeNode *temp = head;
        head = head->next;
        delete temp;
        size--;
        return true;
    }

    EmployeeNode *current = head;
    while (current->next != nullptr) {
        if (current->next->data.getId() == id) {
            EmployeeNode *temp = current->next;
            current->next = temp->next;
            delete temp;
            size--;
            return true;
        }
        current = current->next;
    }

    return false;
}
```

● 问题 2:

问题描述: 工资排序后数据显示错乱, 调用 `sortBySalary()` 后, 控制台输出的工资顺序正确, 但工号与姓名对应关系错误。

```
1001 张三 5000
1002 李四 8000
1003 王五 3000
```

排序前数据

```
1001 王五 3000
1002 张三 5000
1003 李四 8000
```

排序后数据

解决方法: 采用更安全的排序实现方式: 将链表数据转存到临时容器; 在容器内完成排序; 清空原链表; 按顺序重新插入数据。具体步骤为: 使用 `vector` 暂存所有 `Employee` 对象, 调用 STL 的 `sort` 算法排序彻底清空原链表 (释放所有节点), 按排序结果重建链表。

```
void EmployeeList::sortBySalary() {
    if (head == nullptr || head->next == nullptr) return;

    vector<Employee> employees;
    EmployeeNode *current = head;
    while (current != nullptr) {
        employees.push_back(current->data);
        current = current->next;
    }

    sort(employees.begin(), employees.end());

    clear();

    for (const auto &emp : employees) {
        addEmployee(emp);
    }
}
```

● 问题 3:

问题描述: 科室统计结果重复。输出结果包含重复科室条目

```
// 内层循环导致重复统计
EmployeeNode* temp = head;
while(temp) {
    if(temp->data.getDepartment() == dept) {
        count++;
        total += temp->data.getSalary();
    }
    temp = temp->next;
}
cout << dept << "平均工资:" << total/count << endl;
current = current->next; // 导致外层循环重复处理相同科室
}
```

解决方法：首先创建一个字符串数组用于存储已经统计过的科室名称，然后开始遍历链表，对于每个节点所在的科室，先检查该科室名称是否已经存在于已处理科室列表中。如果不存在，则进行完整的科室统计：再次遍历整个链表，计算该科室的总人数和工资总额，并输出平均工资结果，最后将该科室名称加入已处理列表；如果发现该科室已经处理过，则直接跳过。

```
// 收集所有科室
vector<string> departments;
EmployeeNode *current = head;
while (current != nullptr) {
    if (find(departments.begin(), departments.end(), current->data.getDepartment()) == departments.end()) {
        departments.push_back(current->data.getDepartment());
    }
    current = current->next;
}
```

五、 实验总结

本实验通过 C++ 语言开发了一套基于单向链表的职工工资管理系统，圆满完成了课程设计的各项要求。系统采用面向对象的设计思想，通过 Employee 类封装职工基本信息，包括姓名、工号、联系方式等核心字段，并内置了完善的数据校验机制。EmployeeNode 类作为链表节点，将职工数据与指针域有机结合，构建起整个系统的数据结构基础。最核心的 EmployeeList 类负责管理链表操作，实现了从文件加载数据到内存链表的初始化过程，以及各种业务操作后的实时数据持久化。这种分层架构设计使得系统各模块职责明确，耦合度低，便于后续功能扩展和维护。

系统最大的技术亮点在于完整实现了自定义链表结构，严格遵循了“不采用 STL 容器”的要求。在链表操作方面，实现了包括头部/尾部插入、节点删除、遍历查询等基础功能，特别是通过工号精确查询的时间复杂度优化到 $O(1)$ （命中头节点时）。文件持久化模块采用二进制格式存储，通过重载读写操作符实现了 Employee 对象的序列化与反序列化，保证了数据存取的效率和准确性。统计排序功能则巧妙结合了链表的灵活性和 STL 算法的高效性，先将链表数据转存 vector 排序后再重建有序链表，兼顾了开发效率和执行性能。

在开发过程中也暴露出一些值得改进的问题。性能方面，链表查询的平均时间复杂度为 $O(n)$ ，当职工数量超过万级时响应速度明显下降。功能方面缺少批量操作支持，如批量导入导出、批量删除等实用功能。交互体验上，纯命令行界面不够友好，缺乏分页显示和更直观的数据可视化。异常处理机制也有待加强，特别是文件操作时的错误恢复能力需要提升。这些问题主要是由于开发周期紧张和初期设计考虑不周导致的。

通过本次实验，我获得了宝贵的工程实践经验。首先是对数据结构理论有了更深刻的理解，特别是链表的各种操作实现细节和性能特点。其次是掌握了面向对象编程的实际应用技巧，学会了如何设计高内聚低耦合的类结构。最重要的是培养了系统化思维，认识到一个完整的软件系统除了核心功能外，还需要考虑数据持久化、异常处理、用户交互等多个维度。这些经验对我后续的软件开发学习具有重要指导意义。

从工程实践角度，本次实验让我体会到设计文档的重要性。初期由于设计考虑不周全，导致后期不得不修改数据结构，浪费了不少开发时间。同时也认识到测试环节的关键性，特

别是边界条件的测试，往往能发现许多潜在问题。团队协作方面，版本控制工具的使用极大提高了开发效率，这种经验对今后的团队项目开发很有帮助。

总而言之，本次课程设计不仅使我掌握了链表结构和文件操作的具体实现技术，更重要的是培养了软件开发的系统工程思维。从需求分析、系统设计到编码实现、测试调试的全过程实践，让我对软件开发有了更全面和深入的认识。虽然系统还存在不足，但已达到预期目标，为后续更复杂的项目开发打下了坚实基础。这段开发经历将成为我编程学习道路上的重要里程碑。

六、 附录：源代码